

Aggregation, Joins

CST 363

Topics

- "Summary" Functions
- Aggregation Functions: `GROUP BY` , `HAVING`
- Copy data from a table to another
- Update using Case expression
- Set operations: Union, Intersection, Except
- Joins: **Inner** and **Outer** joins, **Natural** join, **Self** joins

"Summary" Functions

- `COUNT(*)` → number of rows in result set
- `COUNT(col_name)` → number of rows with non-null value for col_name
- `COUNT(DISTINCT col_name)` → number of distinct values of a column
- `SUM(col_name)` → sum of column values
- `AVG(col_name)` → average of column values
- `MIN(col_name)` → minimum value in a column
- `MAX(col_name)` → maximum value in a column

COUNT(*) vs. COUNT(col_name)

users

id	name	advisor_id
1	Olistan	10
2	Ravira	NULL
3	Jorvick	12

- COUNT(*) → 3 (all rows)
- COUNT(advisor_id) → 2 (ignores Ravira's NULL)

Summary Functions – Examples

- Example: find the min, max and average instructor salary

```
SELECT MIN(salary), MAX(salary), AVG(salary)
FROM instructor;
```

- Example: how many Comp. Sci. instructors earn \$70,000 or more?

```
SELECT COUNT(*)
FROM instructor
WHERE dept_name = 'Comp. Sci' AND salary ≥ 70000;
```

Aside: Introducing Aliases

Column Aliases

- You can rename a column in the result set using `AS`
 - `SELECT first_name AS fname, last_name AS lname FROM employees;`
 - Result set headers will show fname and lname instead of first_name and last_name

Aliasing Expressions

- Can alias computed values
- `SELECT COUNT(*) AS total_employees FROM employees;`

Aggregate Functions – GROUP BY

- `GROUP BY` is used to aggregate rows that have the same values in one or more columns into summary rows.
- Most often paired with aggregate functions like `COUNT()`, `SUM()`, `AVG()`, `MIN()`, and `MAX()`
- Rows are grouped based on specified column(s), then for each group aggregates are computed
- Every column in the `SELECT` list must either:
 - be in the `GROUP BY` clause, **or**
 - be used inside an aggregate function.

Example: GROUP BY

Find the average salary of instructors in each department:

```
SELECT dept_name, AVG(salary) AS avg_salary
FROM instructor
GROUP BY dept_name;
```

<i>ID</i>	<i>name</i>	<i>dept_name</i>	<i>salary</i>
76766	Crick	Biology	72000
45565	Katz	Comp. Sci.	75000
10101	Srinivasan	Comp. Sci.	65000
83821	Brandt	Comp. Sci.	92000
98345	Kim	Elec. Eng.	80000
12121	Wu	Finance	90000
76543	Singh	Finance	80000
32343	El Said	History	60000
58583	Califieri	History	62000
15151	Mozart	Music	40000
33456	Gold	Physics	87000
22222	Einstein	Physics	95000

<i>dept_name</i>	<i>avg_salary</i>
Biology	72000
Comp. Sci.	77333
Elec. Eng.	80000
Finance	85000
History	61000
Music	40000
Physics	91000

Example: GROUP BY error

- Attributes in `SELECT` clause outside of aggregate functions must appear in `GROUP BY` list

```
-- Erroneous Query
SELECT dept_name, instructor_id, AVG(salary)
FROM instructor
GROUP BY dept_name;
```

A group of instructor rows by `dept_name` has multiple `id` values. So which `id` value will be returned for the group?

Aggregate Functions – HAVING Clause

- `HAVING` is a filter for groups which filters after grouping/aggregation.
- `WHERE` filters rows before grouping/aggregation.

1. Take all rows.
2. Apply `WHERE` (filter rows).
3. Combine rows into groups with `GROUP BY`.
4. Calculate aggregates (`SUM` , `COUNT` , etc.).
5. Apply `HAVING` (filter groups).
6. Return results.

Example: HAVING Clause

- Find the names and average salaries of all departments whose average salary is greater than 42000

```
SELECT dept_name, AVG(salary) AS avg_salary  
FROM instructor  
GROUP BY dept_name  
HAVING AVG(salary) > 42000;
```

- Remember: predicates in the **HAVING** clause are applied after the formation of groups whereas predicates in the **WHERE** clause are applied before forming groups

Copy data from a table to another

- Make each student in the Biology department who has earned more than 100 credit hours an instructor in the Biology department with a salary of \$30,000.

```
INSERT INTO instructor (instructor_id, instructor_name, dept_name, salary)
SELECT student_id, student_name, dept_name, 30_000
FROM student
WHERE dept_name = 'Biology' AND tot_cred > 100;
```

- The `SELECT ... FROM ... WHERE` statement is evaluated fully before any of its results are inserted into the relation.

Updates

- Increase salaries of instructors whose salary is over \$100,000 by 3%, and all others by a 5%

Write two update statements:

```
UPDATE instructor
SET salary = salary * 1.03
WHERE salary > 100000;
```

```
UPDATE instructor
SET salary = salary * 1.05
WHERE salary ≤ 100000;
```

- The order is important
- Can be done better using the `CASE` statement (next slide)

CASE Statement for Conditional Updates

- Can use a CASE expression inside an UPDATE to conditionally set column values.

```
UPDATE table_name
SET column_name = CASE
    WHEN condition1 THEN result1
    WHEN condition2 THEN result2
    ELSE default_result
END
WHERE <filter_condition>;
```

Example: CASE Statement for Conditional Updates

- Same query as before but with case statement

```
UPDATE instructor
SET salary = CASE
    WHEN salary ≤ 100000 THEN salary * 1.05
    ELSE salary * 1.03
END;
```

Set Operations

- Find courses that ran in Fall 2009 or in Spring 2010

```
(  
    SELECT course_id  
    FROM section  
    WHERE semester = 'Fall' AND section_year = 2009  
)  
UNION  
(  
    SELECT course_id  
    FROM section  
    WHERE semester = 'Spring' AND section_year = 2010  
);
```

Set Operations (cont.)

Find courses that ran in Fall 2009 and in Spring 2010

```
(
  SELECT course_id
  FROM section
  WHERE semester = 'Fall' AND section_year = 2009
)
INTERSECT
(
  SELECT course_id
  FROM section
  WHERE semester = 'Spring' AND section_year = 2010
);
```

Find courses that ran in Fall 2009 but not in Spring 2010

```
(
  SELECT course_id
  FROM section
  WHERE semester = 'Fall' AND section_year = 2009
)
EXCEPT
(
  SELECT course_id
  FROM section
  WHERE semester = 'Spring' AND section_year = 2010
);
```

SELECT statements must return the same number of columns and compatible data types for set operations.

Set Operations (cont.)

- Set Operations: `UNION` , `INTERSECT` , and `EXCEPT`
 - Each of the above operations eliminates duplicates.
- To retain all duplicates, use the following:
 - `UNION ALL`
 - `INTERSECT ALL`
 - `EXCEPT ALL`

Joined Relations

- **Join operations** take two relations and return as a result another relation.
- A join operation is based on *Cartesian* product which requires that tuples in the two relations match (specified by predicates).
- The join operations are used as expressions in the `FROM` clause

Cartesian Product

Table One

X	A
1	a
4	d
2	b

3 rows

Table two

X	B
2	x
3	y
5	v

3 rows

Result Set

X	A	X	B
1	a	2	x
1	a	3	y
1	a	5	v
4	d	2	x
4	d	3	y
4	d	5	v
2	b	2	x
2	b	3	y
2	b	5	v

Types of JOIN operations

```
<table name> INNER JOIN <table name> ON <predicates>
```

- Standard explicit join.
- keyword `INNER` is optional

```
<table name> NATURAL JOIN <table name>
```

- Joins automatically on all columns with the same name in both tables.
- Can be convenient, but dangerous if new columns with the same name are added later.

```
<table name> INNER JOIN <table name> USING <column names>
```

- Shorthand when joining on identically named columns.

Basic Query Structure

- A query over multiple tables has the form

```
SELECT A1, A2, ... , An  
FROM R1  
JOIN R2 ON P  
JOIN R3 ON P ...  
WHERE P  
ORDER BY A1, ... , Am
```

- A_i represents an attribute
- R represents a relation
- P is a predicate

Aside: Table Aliases

- You can give a shorter name to a table, useful in joins:

```
SELECT e.first_name, d.dept_name  
FROM employees e  
JOIN departments d ON e.dept_id = d.dept_id;
```

How joins work

instructor

id	name
1	Boris
2	Lena
3	Svetlana

course

course	id	time
DB	1	10
OS	2	4

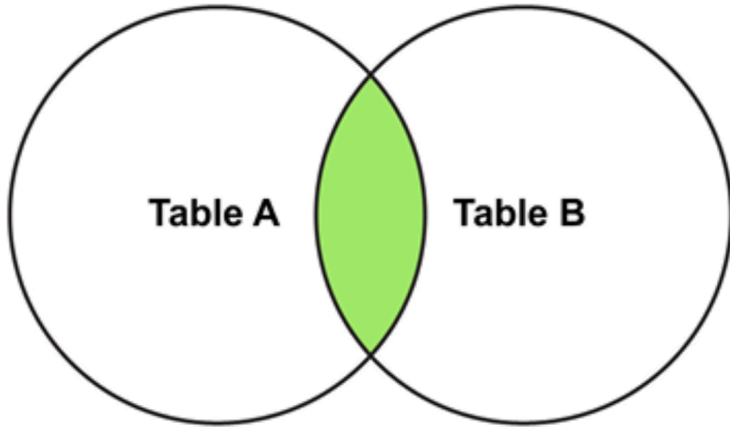
```
SELECT i.name, c.course
FROM instructor i
INNER JOIN course c ON i.id = c.id;
```

id	name	course	id	time
1	Boris	DB	1	10
2	Lena	DB	1	10
3	Svetlana	DB	1	10
1	Boris	OS	2	4
2	Lena	OS	2	4
3	Svetlana	OS	2	4

1. Cartesian product
2. Selection (σ)
3. Projection (π)

Inner Join Visualized

- A standard inner join only includes rows where there is a match in both tables
 - If no match is found for a row in one table, that row is excluded from the result



Example

student JOIN takes on student.ID=takes.ID

student

ID	name	dept_name	tot_cred
00128	Zhang	Comp. Sci.	102
12345	Shankar	Comp. Sci.	32
19991	Brandt	History	80
23121	Chavez	Finance	110
44553	Peltier	Physics	56
45678	Levy	Physics	46
54321	Williams	Comp. Sci.	54
55739	Sanchez	Music	38
70557	Snow	Physics	0
76543	Brown	Comp. Sci.	58
76653	Aoi	Elec. Eng.	60
98765	Bourikas	Elec. Eng.	98
98988	Tanaka	Biology	120

takes

ID	course_id	sec_id	semester	year	grade
00128	CS-101	1	Fall	2017	A
00128	CS-347	1	Fall	2017	A-
12345	CS-101	1	Fall	2017	C
12345	CS-190	2	Spring	2017	A
12345	CS-315	1	Spring	2018	A
12345	CS-347	1	Fall	2017	A
19991	HIS-351	1	Spring	2018	B
23121	FIN-201	1	Spring	2018	C+
44553	PHY-101	1	Fall	2017	B-
45678	CS-101	1	Fall	2017	F
45678	CS-101	1	Spring	2018	B+
45678	CS-319	1	Spring	2018	B
54321	CS-101	1	Fall	2017	A-
54321	CS-190	2	Spring	2017	B+
55739	MU-199	1	Spring	2018	A-
76543	CS-101	1	Fall	2017	A
76543	CS-319	2	Spring	2018	A
76653	EE-181	1	Spring	2017	C
98765	CS-101	1	Fall	2017	C-
98765	CS-315	1	Spring	2018	B
98988	BIO-101	1	Summer	2017	A
98988	BIO-301	1	Summer	2018	null

Example (cont.)

student JOIN takes on student.ID=takes.ID

ID	name	dept_name	tot_cred	course_id	sec_id	semester	year	grade
00128	Zhang	Comp. Sci.	102	CS-101	1	Fall	2017	A
00128	Zhang	Comp. Sci.	102	CS-347	1	Fall	2017	A-
12345	Shankar	Comp. Sci.	32	CS-101	1	Fall	2017	C
12345	Shankar	Comp. Sci.	32	CS-190	2	Spring	2017	A
12345	Shankar	Comp. Sci.	32	CS-315	1	Spring	2018	A
12345	Shankar	Comp. Sci.	32	CS-347	1	Fall	2017	A
19991	Brandt	History	80	HIS-351	1	Spring	2018	B
23121	Chavez	Finance	110	FIN-201	1	Spring	2018	C+
44553	Peltier	Physics	56	PHY-101	1	Fall	2017	B-
45678	Levy	Physics	46	CS-101	1	Fall	2017	F
45678	Levy	Physics	46	CS-101	1	Spring	2018	B+
45678	Levy	Physics	46	CS-319	1	Spring	2018	B
54321	Williams	Comp. Sci.	54	CS-101	1	Fall	2017	A-
54321	Williams	Comp. Sci.	54	CS-190	2	Spring	2017	B+
55739	Sanchez	Music	38	MU-199	1	Spring	2018	A-
76543	Brown	Comp. Sci.	58	CS-101	1	Fall	2017	A
76543	Brown	Comp. Sci.	58	CS-319	2	Spring	2018	A
76653	Aoi	Elec. Eng.	60	EE-181	1	Spring	2017	C
98765	Bourikas	Elec. Eng.	98	CS-101	1	Fall	2017	C-
98765	Bourikas	Elec. Eng.	98	CS-315	1	Spring	2018	B
98988	Tanaka	Biology	120	BIO-101	1	Summer	2017	A
98988	Tanaka	Biology	120	BIO-301	1	Summer	2018	null

JOIN Condition

- The `ON` condition allows a general predicate over the relations being joined
- This predicate is written like a where clause predicate except for the use of the keyword `ON`
- Query example:

```
SELECT *  
FROM student  
JOIN takes ON student.student_id = takes.student_id;
```

- The `ON` condition above specifies that a tuple from student matches a tuple from takes when their `id` values are equal.
- Since the `id` column appears in both tables, it is necessary to qualify it with the table name.
- If there is no match, (i.e. the student has not taken any courses yet), the student row does not show up in the result.

Natural Join

- Natural join matches tuples with the same values for all common attributes, and retains only one copy of each common column.
- List the names of students along with the course ID of the courses that they have taken

```
SELECT student_name, course_id
FROM student
JOIN takes on student.student_id = takes.student_id
```

- Same query in SQL with “natural join” construct

```
SELECT student_name, course_id
FROM student NATURAL JOIN takes;
```


Natural Join (cont.)

- The FROM clause can have multiple relations combined using `NATURAL JOIN`

```
SELECT A1, A2, ... , An
FROM r1
NATURAL JOIN r2 NATURAL JOIN ... NATURAL JOIN rn
WHERE P;
```

Danger in Natural Join

- Beware of unrelated attributes with same name which get equated incorrectly
- Example – List the names of students along with the titles of courses that they have taken

```
--- Correct
SELECT student_name, course.title
FROM student
NATURAL JOIN takes
JOIN course ON takes.course_id = course.course_id;
```

```
--- Incorrect
SELECT student_name, course.title AS course_title
FROM student
NATURAL JOIN takes
NATURAL JOIN course;
```

- The natural join will include the predicate `student.dept_name = course.dept_name`
- This changes the meaning of the query to courses taken in the student's major.

Join with USING Clause

- Query example

```
SELECT student, course
FROM student
NATURAL JOIN takes
JOIN course USING (course_id);
```

- Equivalent to

```
SELECT student, course.title
FROM student
JOIN takes ON student.student_id = takes.student_id
JOIN course ON course.course_id = takes.course_id;
```

Outer Join

- An extension of the join operation that avoids loss of information.
- Computes the join and then adds tuples from one relation that do not match tuples in the other relation to the result of the join.
- Uses null values for missing columns.
- Three forms of outer join:
 - `LEFT JOIN` - adds tuples from the left relation
 - `RIGHT JOIN` - adds tuples from the right relation
 - `FULL JOIN` - adds tuples from both relations

Outer Join Examples

- Relation `course`

<i>course_id</i>	<i>title</i>	<i>dept_name</i>	<i>credits</i>
BIO-301	Genetics	Biology	4
CS-190	Game Design	Comp. Sci.	4
CS-315	Robotics	Comp. Sci.	3

- Relation `prereq`

<i>course_id</i>	<i>prereq_id</i>
BIO-301	BIO-101
CS-190	CS-101
CS-347	CS-101

- Observe that
 - course information is missing CS-437
 - prereq information is missing CS-315

Left Outer Join

```
SELECT *  
FROM course  
LEFT JOIN prereq ON course.course_id = prereq.course_id;
```

<i>course_id</i>	<i>title</i>	<i>dept_name</i>	<i>credits</i>	<i>prereq_id</i>
BIO-301	Genetics	Biology	4	BIO-101
CS-190	Game Design	Comp. Sci.	4	CS-101
CS-315	Robotics	Comp. Sci.	3	<i>null</i>

Right Outer Join

```
SELECT *  
FROM course  
RIGHT OUTER JOIN prereq  
ON course.course_id = prereq.course_id;
```

<i>course_id</i>	<i>title</i>	<i>dept_name</i>	<i>credits</i>	<i>prereq_id</i>
BIO-301	Genetics	Biology	4	BIO-101
CS-190	Game Design	Comp. Sci.	4	CS-101
CS-347	<i>null</i>	<i>null</i>	<i>null</i>	CS-101

Full Outer Join

```
SELECT *  
FROM course  
FULL OUTER JOIN prereq  
ON course.course_id = prereq.course_id;
```

<i>course_id</i>	<i>title</i>	<i>dept_name</i>	<i>credits</i>	<i>prereq_id</i>
BIO-301	Genetics	Biology	4	BIO-101
CS-190	Game Design	Comp. Sci.	4	CS-101
CS-315	Robotics	Comp. Sci.	3	<i>null</i>
CS-347	<i>null</i>	<i>null</i>	<i>null</i>	CS-101

Left Outer Join, Grouping with 0 counts

- Find the number of students each instructor advises. If an instructor does advise any students, report a count of 0.
- Advisor tables has pair of student id, instructor id values.

s_ID	i_ID
12345	10101
44553	22222
45678	22222
00128	45565
76543	45565
23121	76543
98988	76766
76653	98345
98765	98345

- Problem:** Instructor Wu 12121 does not advise any students. An inner join will not find any match for 12121 and so Wu will not be in the result set.
- Solution:** Use left outer join and count column `s_ID`. Left join will return a `s_ID` value of null and count of a null value will give 0.

Left Outer Join, Grouping with 0 counts

```
SELECT instructor.instructor_id, instructor.instructor_name,  
       COUNT(advisor.student_id) AS advised_students  
FROM instructor  
LEFT JOIN advisor ON instructor.instructor_id = advisor.instructor_id  
GROUP BY instructor.instructor_id, instructor.instructor_name  
ORDER BY instructor.instructor_id;
```

ID	name	COUNT(s_ID)
10101	Srinivasan	1
12121	Wu	0
15151	Mozart	0
22222	Einstein	2
32343	El Said	0
33456	Gold	0
45565	Katz	2
58583	Califeri	0
76543	Singh	1
76766	Crick	1
83821	Brandt	0
98345	Kim	2

Self Join Example

```
CREATE TABLE emp_super (  
  person VARCHAR(10) PRIMARY KEY,  
  supervisor VARCHAR(10)  
);
```

- Find Bob's supervisor.
- Find the supervisor of Bob's supervisor.
- Can you find ALL the supervisors (direct and indirect) of Bob?

Self Join and Rename operation

- SQL allows renaming relations and attributes using the AS clause:
 - old-name AS new-name
- Find the names of all instructors who have a higher salary than Wu.

```
SELECT DISTINCT T.name
FROM instructor AS T
JOIN instructor AS S ON T.salary > S.salary
WHERE S.name = 'Wu';
```

Keyword **AS** is optional and may be omitted

```
instructor AS T ≡ instructor T
```

Self Join Example

Find the supervisor of Bob's supervisor.

```
SELECT t1.person, t2.supervisor
FROM emp_super AS t1
JOIN emp_super AS t2 ON t1.supervisor = t2.person
WHERE t1.person = 'Bob';
```

Conclusion – Aggregation and Joins

- **Aggregation** summarizes rows: `COUNT / SUM / AVG / MIN / MAX` with `GROUP BY` .
- Use **WHERE** to filter rows *before* grouping; use **HAVING** to filter groups *after* grouping.
- **Joins** combine tables: **INNER** keeps matches only; **OUTER** preserves unmatched rows with `NULL` .
- Prefer explicit `JOIN ... ON ...` ; use `USING` intentionally; avoid `NATURAL JOIN` unless you fully control shared column names.

Common “gotchas” to avoid

- Selecting non-aggregated columns not in `GROUP BY` .
- Using `COUNT(*)` when you need “0 counts” (use `LEFT JOIN + COUNT(right_table.col)`).
- Accidental joins from same-named columns (the `NATURAL JOIN` trap).